Databases

Final Report: Designing a Telemetry and UX Database for *Chocolate-Doom*

Fergie Lindsay Díaz, Juan Esteban Ortega, Juan Esteban, Federico Mejia

June 2026

Contents

1 Introduction and context

This project is about designing and building a database to store and analyze data collected from a modified version of the game ***Chocolate-Doom***. The game records detailed information during gameplay, such as the player's position, movement, and status at every moment. In addition to this, players also provide basic personal information and complete a user experience survey.

The main goal in the present report or document of this project is to organize all this data in a structured way so it can be stored, accessed, and analyzed efficiently. To do this, we will design a relational database that connects gameplay data with user and survey information. The database should be able to handle a large amount of data and ensure that everything is consistent and well organized.

Another important part of the project is to run queries that help us understand player behavior. For example, we will look at how each player moves, which sectors are more visited, and how individual trajectories develop during the game. We will also relate this information to their survey responses to see how their experience connects to their behavior.

Finally, we will follow good practices for data handling, including cleaning and loading the data properly, and considering basic privacy and ethical aspects. Overall, the project aims to combine database design with practical analysis of real gameplay data.

2 Assumptions and requirements

2.1 Assumptions

- Each User is a real participant who has accepted consent before any data is collected.
- A User can have one or more Player identities (nicknames), but each Player belongs to only one User.
- Each Game represents one gameplay session with a start and end time.
- Each Game is linked to exactly one Player.
- Telemetry data is recorded at every tic (very frequent events) for each game session.
- Each telemetry record is uniquely identified by (game_id, tic).
- Each Game is linked to exactly one Map and one Episode.
- Maps are divided into Sectors, which help us analyze player positions.
- Each user is expected to submit one UX survey after gameplay.
- Survey responses are completed after the game session and linked to the user.

2.2 Functional requirements

- Store and manage user demographic information (age, gender, experience level, consent).
- Support multiple player identities per user.
- Record high-frequency gameplay telemetry including position, momentum, orientation, and combat stats.
- Store game session metadata including player, episode, map, start time, and end time.
- Support spatial grouping of telemetry data by map sectors.
- Ingest raw telemetry logs (TSV format) into a structured relational database.
- Store and manage UX survey instruments (PENS, GUESS, BANGS).
- Store user responses to UX instruments at the question level.
- Support analytical queries on movement, trajectories, and player behavior.
- Support aggregation of telemetry data for trajectory and gameplay behavior analysis.

2.3 Non-functional requirements

- The database should handle large telemetry datasets efficiently (at least 20,000 records, but ideally much more as data grows).
- The schema should work well as the number of players, games, and telemetry records increases, without needing major changes.
- Primary keys and foreign keys should be used to keep data consistent across all tables.
- Indexes should be added to improve performance for common queries, especially those involving time sequences and spatial data.
- The design should allow new telemetry fields or new UX instruments to be added without redesigning the whole database.
- Data loading should be repeatable using a clear ETL process so the database can be rebuilt from raw logs if needed.
- User data should be anonymized, and consent must be stored before any data is collected or used.
- If some telemetry records are invalid or broken, they should be logged and skipped without stopping the whole ingestion process.

3 Survey Election - GUESS

The survey we decided to work with and base our design on is Game User-Experience Satisfaction Scale **GUESS**.

3.1 What is it?

The Game User Experience Satisfaction Scale (GUESS-18) is a survey used to understand how players feel about a video game. It has 18 questions that are divided into nine sections, and each one looks at a different part of the experience, like how easy the game is to use, how fun it is, how immersive it feels, or how good the visuals and audio are.

Each question is answered using a scale from 1 to 7, where players say how much they agree or disagree. At the end, the answers are grouped to get scores for each section and also a total score that shows the overall satisfaction. There is one question that is reverse scored ("I feel bored while playing the game"), so it needs to be handled a bit differently.

GUESS-18 is actually a shorter version of a longer survey with 55 questions, but this one is faster to complete and still gives good results.

3.2 Why did we chose it?

We chose GUESS-18 because it is easy to use and still gives a lot of useful information. It lets us measure different aspects of the player experience, not just if the game works, but also if it is fun, engaging, or interesting.

Another reason is that it is a well-known and tested survey, so we can trust the results we get. This is important because we want to compare what players say with the data we collect from the game.

It also fits nicely with our database design. Since the survey is divided into sections and questions, we can store everything in an organized way and later analyze things like enjoyment or immersion separately.

Finally, it is short, so players can answer it quickly after playing. This makes it more likely that we get complete and honest responses.

4 Entity-Relation Diagram

The diagram organizes gameplay telemetry, user information, and survey responses into coherent relationships. It highlights how users are linked to players, how players are linked to individual game sessions, and how each session generates detailed telemetry data. It also integrates UX instruments to connect behavioral data with subjective user experience.

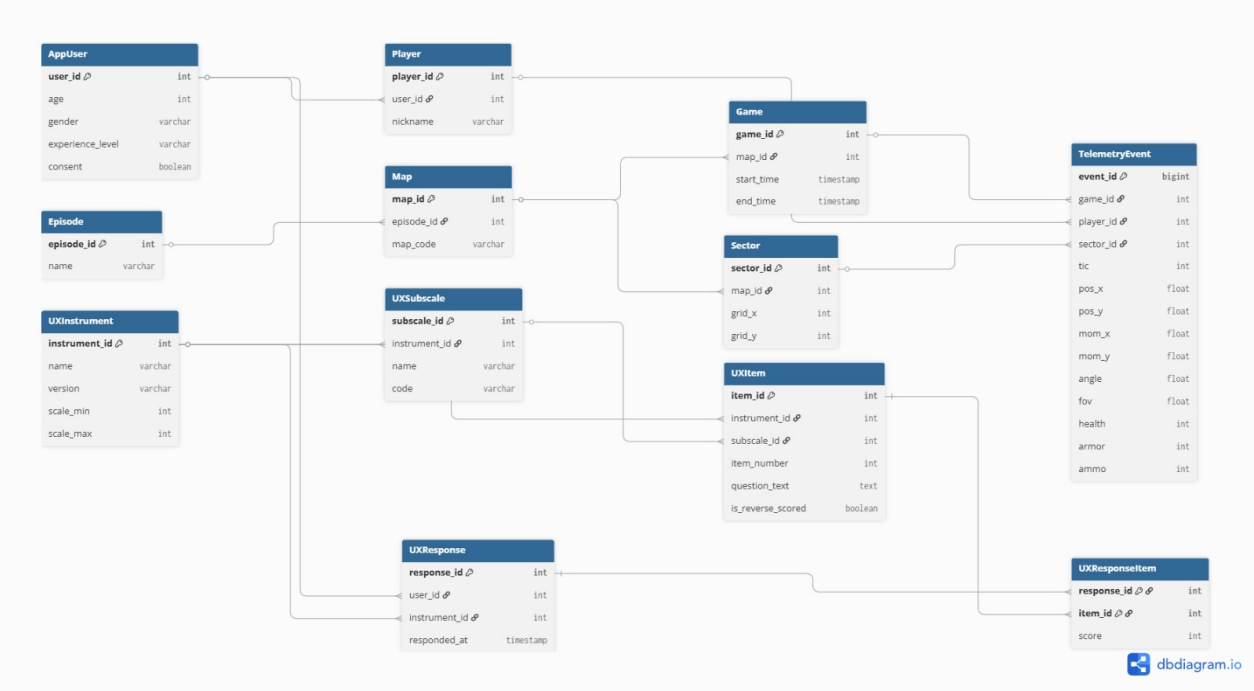


Figure 1: Entity-Relation Diagram

The diagram organizes gameplay telemetry, user information, and survey responses into coherent relationships. It highlights how users are linked to players, how players participate in game sessions, and how each session generates detailed telemetry data. It also integrates UX instruments to connect behavioral data with subjective user experience.

5 Relational Schema

The database is normalized by separating participant identity, gameplay metadata, map structure, telemetry events, and UX responses. High-frequency telemetry is isolated in `TelemetryEvent`, while relatively stable reference data such as `Episode`, `Map`, `Sector`, and UX metadata remain in separate tables.

The most important correction in the final schema is the composite key in `Game`. A simple primary key on `game_id` would fail with the real telemetry because one session can include more than one participant. The final model therefore uses:

```
PRIMARY KEY (game_id, player_id)
```

and `TelemetryEvent` includes `player_id` and references the composite key:

```
FOREIGN KEY (game_id, player_id)
REFERENCES Game(game_id, player_id)
ON DELETE CASCADE
```

5.1 Core entities

This subsection lists the core entities identified in the system. These represent the main objects stored in the database, including users, gameplay sessions, telemetry records, and UX survey components.

- `AppUser`: participant demographics and informed consent.
- `Player`: in-game alias linked to a user.
- `Episode`, `Map`, `Sector`: level hierarchy and spatial grouping.
- `Game`: player participation in a shared session.
- `TelemetryEvent`: atomic per-tic player state.
- `UXInstrument`, `UXSubscale`, `UXItem`: UX questionnaire metadata.
- `UXResponse`, `UXResponseItem`: user responses at response and item level.

5.2 Foreign Keys

The way in which we write or show the foreign keys will later help us in the way we can use it in SQL.

Example:

```
        FOREIGN KEY (user_id) REFERENCES User(user_id)
```

This subsection defines the relationships between tables using foreign keys. These constraints enforce referential integrity and ensure consistency across users, gameplay sessions, telemetry events, and UX responses.

5.3 Data Dictionary

AppUser

Attribute	Type	Constraints	Description
user_id	SERIAL	PK	Unique identifier for each user
age	INTEGER	CHECK (10–100)	Age of the participant
gender	VARCHAR(50)	—	Self-reported gender
experience_level	VARCHAR(50)	—	Level of gaming experience
consent	BOOLEAN	NOT NULL, DEFAULT FALSE	Indicates informed consent for participation

Player

player_id	SERIAL	PK	Unique identifier for each in-game player
user_id	INTEGER	FK, NOT NULL	References AppUser(user_id); links player to a user
nickname	VARCHAR(100)	NOT NULL	Player's in-game alias

Episode

episode_id	SERIAL	PK	Unique episode identifier
name	VARCHAR(100)	NOT NULL	Name of the episode

Map

map_id	SERIAL	PK	Unique map identifier
episode_id	INTEGER	FK	References Episode(episode_id)
map_code	VARCHAR(50)	NOT NULL	Map code (e.g., E1M1)

Sector

sector_id	SERIAL	PK	Unique sector identifier
map_id	INTEGER	FK	References Map(map_id)
grid_x	INTEGER	—	X-coordinate within the map grid
grid_y	INTEGER	—	Y-coordinate within the map grid

Game

game_id	SERIAL	PK	Unique identifier for each game session
player_id	INTEGER	FK	References Player(player id)
map_id	INTEGER	FK	References Map(map_id)
start_time	TIMESTAMP	NOT NULL	Timestamp when the session started
end_time	TIMESTAMP	—	Timestamp when the session ended

TelemetryEvent

event_id	BIGSERIAL	PK	Unique identifier for each telemetry record
game_id	INTEGER	FK	References Game(game_id)
player_id	INTEGER	FK	References Player(player_id)
sector_id	INTEGER	FK	Nullable reference to Sector(sector_id)
tic	INTEGER	NOT NULL	Discrete time unit (game tick)
pos_x	DOUBLE PRECISION	NOT NULL	Player position on X-axis
pos_y	DOUBLE PRECISION	NOT NULL	Player position on Y-axis
mom_x	DOUBLE PRECISION	—	Momentum on X-axis
mom_y	DOUBLE PRECISION	—	Momentum on Y-axis
angle	DOUBLE PRECISION	—	Player orientation angle
fov	DOUBLE PRECISION	—	Field of view
health	INTEGER	—	Player health value
armor	INTEGER	—	Player armor value
ammo	INTEGER	—	Ammunition count

Constraint: UNIQUE (game_id, tic)

UXInstrument

instrument_id	SERIAL	PK	Unique identifier for each UX instrument
name	VARCHAR(100)	NOT NULL	Name of the instrument (e.g., GUESS, PENS)
version	VARCHAR(50)	—	Instrument version
scale_min	INTEGER	—	Minimum possible score
scale_max	INTEGER	—	Maximum possible score

UXSubscale

subscale_id	SERIAL	PK	Unique identifier for each subscale
instrument_id	INTEGER	FK	References UXInstrument(instrument_id)
name	VARCHAR(100)	NOT NULL	Name of the subscale
code	VARCHAR(20)	—	Short code identifier

UXItem

item_id	SERIAL	PK	Unique identifier for each item/question
instrument_id	INTEGER	FK	References UXInstrument(instrument_id)
subscale_id	INTEGER	FK	References UXSubscale(subscale_id)
item_number	INTEGER	NOT NULL	Item order within the instrument
question_text	TEXT	NOT NULL	Text of the survey question
is_reverse_scored	BOOLEAN	DEFAULT FALSE	Indicates if reverse scoring is applied

UXResponse

response_id	SERIAL	PK	Unique identifier for each survey response
user_id	INTEGER	FK	References AppUser(user_id)
instrument_id	INTEGER	FK	References UXInstrument(instrument_id)
responded_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Date and time of submission

UXResponseItem

response_id	INTEGER	PK, FK	References UXResponse(response_id)
item_id	INTEGER	PK, FK	References UXItem(item_id)
score	INTEGER	CHECK (≥ 0)	Score given to the item

6 Data Definition Language (DDL)

6.1 SQL Dictionary

- **SERIAL - BIGSERIAL**: Auto-increment numbers for IDs. *Example:* In `AppUser`, `user_id SERIAL PRIMARY KEY` gives each user a unique ID. In `TelemetryEvent`, `event_id BIGSERIAL PRIMARY KEY` allows billions of events.
- **CHECK**: Rule that makes sure values are valid. *Example:* In `AppUser`, `age INTEGER CHECK (age BETWEEN 10 AND 100)` only allows ages between 10 and 100. In `UXResponseItem`, `score INTEGER CHECK (score >= 0)` ensures scores are not negative.
- **ON DELETE CASCADE**: If a parent row is deleted, linked child rows are deleted too. *Example:* If an *AppUser* is deleted, their *Player* record is also deleted. If a Game is deleted, all its *TelemetryEvent* records are removed.
- **TIMESTAMP**: Stores both date and time. *Example:* In `Game`, `start_time TIMESTAMP NOT NULL` records when the game started. In `UXResponse`, `responded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP` saves the time a user answered.
- **PRECISION**: Controls how exact numbers are stored. *Example:* In `TelemetryEvent`, `pos_x DOUBLE PRECISION` stores very detailed player positions, momentum, and angles.
- **ON DELETE SET NULL**: If a parent row is deleted, the link is removed but the child row stays (foreign key becomes `NULL`). *Example:* If a `Sector` is deleted, the `TelemetryEvent` still exists but its `sector_id` becomes `NULL`. If a `UXSubscale` is deleted, the `UXItem` remains but loses its subscale link.

6.2 Data Definition Language highlights

The DDL was implemented in PostgreSQL. The final schema uses standard relational constraints and keeps telemetry fields typed as numeric values so that spatial and temporal analysis can be performed directly in SQL.

```
-- 6. Game
-- game_id identifica una sesion compartida; varios jugadores participan
-- en la misma sesion, por eso la PK es (game_id, player_id).
CREATE TABLE Game (
    game_id   INTEGER NOT NULL,
    player_id INTEGER NOT NULL REFERENCES Player(player_id) ON DELETE CASCADE,
    map_id    INTEGER REFERENCES Map(map_id) ON DELETE CASCADE,
    start_time TIMESTAMP NOT NULL,
    end_time   TIMESTAMP,
    PRIMARY KEY (game_id, player_id)
);

-- Secuencia separada para generar nuevos game_id
CREATE SEQUENCE game_game_id_seq START 1;

-- 7. TelemetryEvent
CREATE TABLE TelemetryEvent (
    event_id BIGSERIAL PRIMARY KEY,
    game_id   INTEGER NOT NULL,
    player_id INTEGER NOT NULL,
    sector_id INTEGER,  -- ID del sector en el mapa (no FK; los IDs del motor Doom no son secuenciales)
    tic INTEGER NOT NULL,
    pos_x DOUBLE PRECISION NOT NULL,
```

```
    pos_y DOUBLE PRECISION NOT NULL,
    mom_x DOUBLE PRECISION,
    mom_y DOUBLE PRECISION,
    angle DOUBLE PRECISION,
    fov DOUBLE PRECISION,
    health INTEGER,
    armor INTEGER,
    ammo INTEGER,
    CONSTRAINT uq_telem UNIQUE (game_id, player_id, tic),
    CONSTRAINT fk_telem_game FOREIGN KEY (game_id, player_id)
        REFERENCES Game(game_id, player_id) ON DELETE CASCADE
);
```

7 SQL CODES

7.1 parte2_puntoB.sql

In this section, we explain how the telemetry data is loaded into the database.

First, the raw data from the TSV file is stored in the *staging_telemetry* table. In this table, all values are saved as text to make the loading process simple.

Then, we check for invalid records (for example, missing game, player, or tic information) and store them in the *ingestion_error_log* table.

Finally, the valid data is cleaned, converted to the correct data types, and inserted into the `TelemetryEvent` table. Duplicate records are avoided, and empty values are handled properly.

```
-- 1. Crear tabla de Staging
CREATE TABLE IF NOT EXISTS staging_telemetry (
    game_id_raw      TEXT,
    player_id_raw    TEXT,
    sector_id_raw    TEXT,
    tic_raw          TEXT,
    pos_x_raw        TEXT,
    pos_y_raw        TEXT,
    mom_x_raw        TEXT,
    mom_y_raw        TEXT,
    angle_raw        TEXT,
    fov_raw          TEXT,
    health_raw       TEXT,
    armor_raw        TEXT,
    ammo_raw         TEXT
);

-- 2. Crear tabla de Log de Errores
CREATE TABLE IF NOT EXISTS ingestion_error_log (
    error_id   SERIAL PRIMARY KEY,
    game_id    TEXT,
    tic        TEXT,
    motivo     TEXT,
    fecha_err  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Proceso ETL
INSERT INTO ingestion_error_log (game_id, tic, motivo)
SELECT game_id_raw, tic_raw,
FROM staging_telemetry
WHERE game_id_raw IS NULL OR player_id_raw IS NULL OR tic_raw IS NULL;

INSERT INTO TelemetryEvent (
    game_id, player_id, sector_id, tic,
```

```
    pos_x, pos_y,
    mom_x, mom_y,
    angle, fov, health, armor, ammo
)
SELECT DISTINCT ON (CAST(game_id_raw AS INT), CAST(player_id_raw AS INT), CAST(tic_raw AS INT))
    CAST(game_id_raw AS INT),
    CAST(player_id_raw AS INT),
    CAST(NULLIF(sector_id_raw, '') AS INT),
    CAST(tic_raw AS INT),
    CAST(pos_x_raw AS DOUBLE PRECISION),
    CAST(pos_y_raw AS DOUBLE PRECISION),
    CAST(NULLIF(mom_x_raw, '') AS DOUBLE PRECISION),
    CAST(NULLIF(mom_y_raw, '') AS DOUBLE PRECISION),
    CAST(NULLIF(angle_raw, '') AS DOUBLE PRECISION),
    CAST(NULLIF(fov_raw, '') AS DOUBLE PRECISION),
    CAST(NULLIF(health_raw, '') AS INT),
    CAST(NULLIF(armor_raw, '') AS INT),
    CAST(NULLIF(ammo_raw, '') AS INT)
FROM staging_telemetry
WHERE game_id_raw IS NOT NULL
  AND player_id_raw IS NOT NULL
  AND tic_raw IS NOT NULL
ORDER BY CAST(game_id_raw AS INT), CAST(player_id_raw AS INT), CAST(tic_raw AS INT);
```

7.2 parte_3_puntoB.sql

In this section we insert the GUESS-18 survey structure into the database, including the instrument, its subscales, and all items.

The field `is_reverse_scored` is used to indicate whether a question needs reverse scoring:

- **TRUE:** The item is reverse scored. This means that higher values represent a negative response, so the score must be inverted during analysis.
- **FALSE:** The item is not reverse scored. The values can be interpreted directly, where higher scores indicate a more positive experience.

The questionnaire is divided into subscales, which are groups of questions that measure different aspects of the player experience.

Each subscale focuses on a specific category, such as :

- **Usability/Playability** → how easy the game is to play
- **Narratives** → story and engagement with the game's plot
- **Play Engrossment** → level of immersion and focus during gameplay
- **Enjoyment** → how fun the game feels
- **Creative Freedom** → ability to be imaginative while playing
- **Audio Aesthetics** → quality and impact of sound and music
- **Personal Gratification** → focus on performance and achievement
- **Visual Aesthetics** → visual quality and appeal of the game

This organization allows the analysis to evaluate different dimensions of the game experience separately, instead of treating all responses as a single score.

In our implementation, the GUESS-18 survey is divided into nine subscales, each containing two items or questions, but we used **8 / 9** because one of these included the experience with other players, what was discussed and taken out to follow instructions and rules of the game: one player.

Each question (item) belongs to one subscale, allowing us to group responses and analyze different aspects of the player experience independently.

Each subscale is stored in the `UXSubscale` table and is linked to its corresponding questions through the `UXItem` table. This structure allows efficient organization and analysis of survey data.

- Subscale = Topic of analysis for the survey.
- Item = Question

```sql
-- 1. Insertar el instrumento GUESS-18
INSERT INTO UXInstrument (instrument_id, name, version, scale_min, scale_max)
VALUES (1, 'GUESS', '18-Item Short Scale', 1, 7);

-- 2. Insertar 8/9 Subescalas oficiales del GUESS-18
INSERT INTO UXSubscale (subscale_id, instrument_id, name, code) VALUES
(1, 1, 'Usability/Playability', 'USAB'),
(2, 1, 'Narratives', 'NARR'),
(3, 1, 'Play Engrossment', 'ENGR'),
(4, 1, 'Enjoyment', 'ENJO'),
(5, 1, 'Creative Freedom', 'CREA'),
(6, 1, 'Audio Aesthetics', 'AUDI'),
(7, 1, 'Personal Gratification', 'GRAT'),
(8, 1, 'Visual Aesthetics', 'VISU');

-- 3. Insertar los 16 items (2 por subescala)
INSERT INTO UXItem (item_id, instrument_id, subscale_id, item_number, question_text, is_reverse_scored) VALUES
-- Usability/Playability
(1, 1, 1, 1, 'I find the controls of the game to be straightforward.', FALSE),
(2, 1, 1, 2, 'I find the game''s interface to be easy to navigate.', FALSE),

-- Narratives
(3, 1, 2, 3, 'I am captivated by the game''s story from the beginning.', FALSE),
(4, 1, 2, 4, 'I enjoy the fantasy or story provided by the game.', FALSE),

-- Play Engrossment
(5, 1, 3, 5, 'I feel detached from the outside world while playing the game.', FALSE),
(6, 1, 3, 6, 'I do not care to check events that are happening in the real world during the game.', FALSE),

-- Enjoyment
(7, 1, 4, 7, 'I think the game is fun.', FALSE),
(8, 1, 4, 8, 'I feel bored while playing the game.', TRUE),

-- Creative Freedom
(9, 1, 5, 9, 'I feel the game allows me to be imaginative.', FALSE),
(10, 1, 5, 10, 'I feel creative while playing the game.', FALSE),

-- Audio Aesthetics
(11, 1, 6, 11, 'I enjoy the sound effects in the game.', FALSE),
(12, 1, 6, 12, 'I feel the game''s audio enhances my gaming experience.', FALSE),

-- Personal Gratification
(13, 1, 7, 13, 'I am very focused on my own performance while playing the game.', FALSE),
(14, 1, 7, 14, 'I want to do as well as possible during the game.', FALSE),

-- Visual Aesthetics
(15, 1, 8, 15, 'I enjoy the game''s graphics.', FALSE),
(16, 1, 8, 16, 'I think the game is visually appealing.', FALSE);

-- Resincronizar las secuencias en PostgreSQL para futuros INSERTS
SELECT setval('uxinstrument_instrument_id_seq', (SELECT MAX(instrument_id) FROM UXInstrument));
SELECT setval('uxsubscale_subscale_id_seq', (SELECT MAX(subscale_id) FROM UXSubscale));
SELECT setval('uxitem_item_id_seq', (SELECT MAX(item_id) FROM UXItem));
```

8 C.3 Views and Materialized View

This section defines SQL views and a materialized view used to support frequent analytical queries over gameplay and telemetry data. These structures simplify complex queries and improve performance for repeated analyses.

8.1 View 1: Player Game Summary

```sql
CREATE VIEW player_game_summary AS
SELECT
    p.player_id,
    p.nickname,
    COUNT(g.game_id) AS total_games,
    MIN(g.start_time) AS first_game,
    MAX(g.end_time) AS last_game
FROM Player p
JOIN Game g ON p.player_id = g.player_id
GROUP BY p.player_id, p.nickname;
```

This view summarizes the activity of each player by counting the number of games played and showing their first and last recorded sessions.

8.2 View 2: Map and Episode Activity Summary

```sql
CREATE VIEW map_activity_summary AS
SELECT
    e.episode_id,
    e.name AS episode_name,
    m.map_id,
    m.map_code,
    COUNT(g.game_id) AS total_games
FROM Episode e
JOIN Map m ON e.episode_id = m.episode_id
JOIN Game g ON m.map_id = g.map_id
GROUP BY e.episode_id, e.name, m.map_id, m.map_code;
```

This view provides a summary of gameplay activity per map and episode, allowing analysis of which levels are most frequently played.

8.3 Materialized View: Player Telemetry Summary

```sql
CREATE MATERIALIZED VIEW player_telemetry_summary AS
SELECT
    g.game_id,
    g.player_id,
    COUNT(t.event_id) AS total_events,
    AVG(t.pos_x) AS avg_pos_x,
    AVG(t.pos_y) AS avg_pos_y,
    AVG(t.health) FILTER (WHERE t.health IS NOT NULL) AS avg_health,
    AVG(t.ammo) FILTER (WHERE t.ammo IS NOT NULL) AS avg_ammo
FROM Game g
JOIN TelemetryEvent t ON g.game_id = t.game_id
GROUP BY g.game_id, g.player_id;
```

This materialized view precomputes aggregated telemetry statistics per game session. Since telemetry data is large and frequently queried, storing pre-aggregated results improves performance significantly.

A materialized view is used instead of a regular view because regular views execute the query every time they are called. In contrast, the materialized view saves the results, making access faster. However, this comes with the limitation that the data must be manually refreshed whenever new telemetry data is added.

8.4 Refreshing the Materialized View

```
REFRESH MATERIALIZED VIEW player_telemetry_summary;
```

This command is used to update the materialized view when new telemetry data is inserted into the database.

9 Analytical queries and results

This section presents six analytical queries selected from the project brief. They were adapted to the corrected schema, where a shared session is represented by `game_id` and each player participation is represented by `(game_id, player_id)`.

9.1 Query 1: average session duration per map

This query groups games by map and calculates how long each session lasts using the start and end times. Then, it computes the average duration for each map.

Combines the tables `Game`, `Map`, and `Episode` using `JOIN`, so each game session is linked to its map and episode. The `WHERE` clause keeps only finished sessions (those with `end_time`). Then, `GROUP BY` organizes the data per map, and aggregate functions like `COUNT` and `AVG` are used to calculate how many sessions there are and their average duration.

```sql
SELECT
    m.map_id,
    m.map_code,
    e.name AS episodio,
    COUNT(*) AS sesiones,
    AVG(g.end_time - g.start_time) AS duracion_promedio
FROM Game g
JOIN Map m     ON g.map_id = m.map_id
JOIN Episode e ON m.episode_id = e.episode_id
WHERE g.end_time IS NOT NULL
GROUP BY m.map_id, m.map_code, e.name
ORDER BY duracion_promedio DESC;
```

```
 map_id | map_code |       episodio        | sesiones | duracion_promedio
--------+----------+-----------------------+----------+-------------------
      1 | E1M1     | Knee-Deep in the Dead |        5 | 00:00:40.4
      2 | E1M2     | Knee-Deep in the Dead |        5 | 00:00:33
      3 | E2M1     | The Shores of Hell    |        5 | 00:00:31
      4 | E2M2     | The Shores of Hell    |        5 | 00:00:29.8
      5 | E3M1     | Inferno               |        4 | 00:00:29
      6 | E3M2     | Inferno               |        4 | 00:00:26
(6 rows)
```

Figure 2: Average session duration by map.

Interpretation. This helps us see which maps tend to keep players playing for longer. For example, a map with longer sessions might be more engaging or take more time to complete.

9.2 Query 2: players with the highest average proximity

This query compares players by looking at their positions at the same *tic*, even if they played in different sessions. It calculates the average distance between them.

This query compares players using a self-join, (`TelemetryEvent` joined with itself). The condition `a.tic = b.tic` matches positions at the same moment in time, and `a.player_id < b.player_id` avoids duplicating pairs. The distance between positions is calculated using `sqrt` and `power`, and then averaged using `AVG`. Finally, the results are grouped by pairs of players.

```
SELECT
    a.player_id AS jugador_a,
    b.player_id AS jugador_b,
    COUNT(*) AS tics_comparados,
    ROUND(
        AVG(
            sqrt(
                power(a.pos_x - b.pos_x, 2) +
                power(a.pos_y - b.pos_y, 2)
            )
        )::numeric, 2
    ) AS distancia_promedio
FROM TelemetryEvent a
JOIN TelemetryEvent b
  ON a.tic = b.tic
 AND a.player_id < b.player_id
GROUP BY a.player_id, b.player_id
ORDER BY distancia_promedio ASC;
```

jugador_a	jugador_b	tics_comparados	distancia_promedio
5	6	18200	1622.00
1	6	21300	1632.48
2	5	22100	1722.99
2	6	21800	1748.62
2	4	25800	1753.07
3	6	21000	1795.85
4	6	20700	1801.44
1	5	21400	1824.97
4	5	20900	1861.26
2	3	26000	1872.60
1	2	26000	1882.81
3	4	25600	1888.12
1	4	25200	1915.95
1	3	25400	1944.73
3	5	21200	2116.04

(15 rows)

Figure 3: Closest player pairs by average distance.

Interpretation. Since the game is single-player, this does not represent real interaction. Instead, we imagine all players moving at the same time.

If the average distance is small, it means players tend to follow similar paths in the game.

9.3 Query 3: shortest and longest trajectory distances per player

This query uses a window function called `LAG()` to access the previous position of a player. The data is partitioned by `game_id` and `player_id`, so each player's movement is analyzed within their own session. Then, the distance between consecutive positions is calculated and summed using `SUM`. Finally, `GROUP BY` is used to compute the minimum and maximum trajectory distance per player.

```sql
WITH pasos AS (
    SELECT
        game_id,
        player_id,
        tic,
        pos_x,
        pos_y,
        LAG(pos_x) OVER (PARTITION BY game_id, player_id ORDER BY tic) AS px_prev,
        LAG(pos_y) OVER (PARTITION BY game_id, player_id ORDER BY tic) AS py_prev
    FROM TelemetryEvent
),
dist_por_partida AS (
    SELECT
        game_id,
        player_id,
        SUM(
            sqrt(
                power(pos_x - px_prev, 2) +
                power(pos_y - py_prev, 2)
            )
        ) AS dist_total
    FROM pasos
    WHERE px_prev IS NOT NULL
    GROUP BY game_id, player_id
)
SELECT
    player_id,
    COUNT(*) AS partidas_jugadas,
    ROUND(MIN(dist_total)::numeric, 2) AS trayectoria_min,
    ROUND(MAX(dist_total)::numeric, 2) AS trayectoria_max
FROM dist_por_partida
GROUP BY player_id
ORDER BY player_id;
```

player_id	partidas_jugadas	trayectoria_min	trayectoria_max
1	5	7587.75	17487.44
2	5	8448.40	15889.34
3	5	6060.07	11384.56
4	5	5843.17	8502.11
5	4	9838.55	18994.42
6	4	6488.90	14039.98

(6 rows)

Figure 4: Minimum and maximum trajectory distance by player.

Interpretation. Trajectory length provides a quantitative measure of player movement. Large trajectory values suggest exploration-oriented behavior, while shorter trajectories may indicate defensive or objective-focused play styles.

9.4 Query 4: UX responses for players with above-average trajectory duration

This query uses two common table expressions (CTEs). The first one counts how many tics each player has using `COUNT`. The second calculates the global average using `AVG`. Then, the query selects players above the average and joins several tables (`Player`, `UXResponse`, `UXItem`, and `UXSubscale`). Finally, it calculates the average score per subscale using `AVG`.

```
WITH duracion AS (
    SELECT
        player_id,
        COUNT(*) AS tics_totales
    FROM TelemetryEvent
    GROUP BY player_id
),
promedio AS (
    SELECT AVG(tics_totales) AS media_global FROM duracion
)
SELECT
    p.player_id,
    p.nickname,
    d.tics_totales,
    s.code AS subescala,
    ROUND(AVG(ri.score), 2) AS puntaje_promedio
FROM duracion d
CROSS JOIN promedio pr
JOIN Player p          ON p.player_id = d.player_id
JOIN UXResponse r      ON r.user_id = p.user_id
JOIN UXResponseItem ri ON ri.response_id = r.response_id
JOIN UXItem i          ON i.item_id = ri.item_id
JOIN UXSubscale s      ON s.subscale_id = i.subscale_id
WHERE d.tics_totales > pr.media_global
GROUP BY p.player_id, p.nickname, d.tics_totales, s.code
ORDER BY p.player_id, s.code;
```

player_id	nickname	tics_totales	subescala	puntaje_promedio
1	Doomguy	6000	AUDI	6.00
1	Doomguy	6000	CREA	5.00
1	Doomguy	6000	ENGR	4.50
1	Doomguy	6000	ENJO	4.00
1	Doomguy	6000	GRAT	5.00
1	Doomguy	6000	NARR	5.00
1	Doomguy	6000	USAB	5.00
1	Doomguy	6000	VISU	5.50
2	Ranger	6300	AUDI	5.00
2	Ranger	6300	CREA	4.00
2	Ranger	6300	ENGR	4.00
2	Ranger	6300	ENJO	4.00
2	Ranger	6300	GRAT	5.00
2	Ranger	6300	NARR	3.00
2	Ranger	6300	USAB	3.00
2	Ranger	6300	VISU	3.50
3	Crash	5800	AUDI	5.50
3	Crash	5800	CREA	5.50
3	Crash	5800	ENGR	6.00
3	Crash	5800	ENJO	6.00
3	Crash	5800	GRAT	5.00
3	Crash	5800	NARR	5.50
3	Crash	5800	USAB	6.50
3	Crash	5800	VISU	5.50

(24 rows)

Figure 5: Players above the global average trajectory duration.

Interpretation. By combining UX responses with trajectory duration, this query explores whether players who remain active longer report different subjective experiences than average players.

9.5 Query 5: most visited sector per episode and map

This query counts how many times each sector is visited using `COUNT(*)`. The data is grouped by episode, map, and sector. Then, the window function `ROW_NUMBER()` is used to rank sectors based on the number of visits within each map. Finally, only the top-ranked sector (the most visited one) is selected.

```sql
WITH conteo AS (
    SELECT
        e.episode_id,
        e.name AS episodio,
        m.map_id,
        m.map_code,
        t.sector_id,
        COUNT(*) AS visitas,
        ROW_NUMBER() OVER (
            PARTITION BY e.episode_id, m.map_id
            ORDER BY COUNT(*) DESC
        ) AS rn
    FROM TelemetryEvent t
    JOIN Game g
      ON g.game_id = t.game_id
     AND g.player_id = t.player_id
    JOIN Map m     ON m.map_id = g.map_id
    JOIN Episode e ON e.episode_id = m.episode_id
    WHERE t.sector_id IS NOT NULL
    GROUP BY e.episode_id, e.name, m.map_id, m.map_code, t.sector_id
)
SELECT
    episodio,
    map_code,
    sector_id AS sector_hotspot,
    visitas
FROM conteo
WHERE rn = 1
ORDER BY episodio, map_code;
```

```
       episodio        | map_code | sector_hotspot | visitas
-----------------------+----------+----------------+---------
 Inferno               | E3M1     |            246 |     217
 Inferno               | E3M2     |            707 |     163
 Knee-Deep in the Dead | E1M1     |             15 |     268
 Knee-Deep in the Dead | E1M2     |            655 |     293
 The Shores of Hell    | E2M1     |            862 |     158
 The Shores of Hell    | E2M2     |            489 |     194
(6 rows)
```

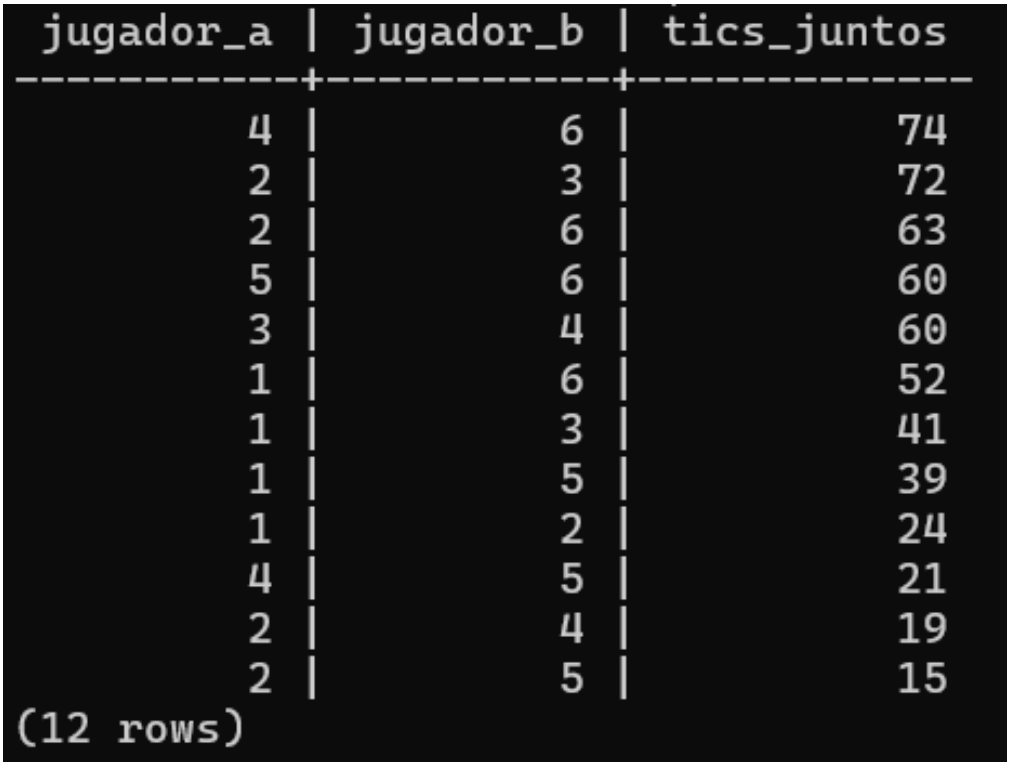
Figure 6: Most visited sector by episode and map.

Interpretation. Hotspot sectors reveal the areas that attract the highest player activity. These sectors may correspond to combat zones, resource-rich areas, or strategic map locations.

9.6 Query 6: number of tics where players were together in a sector

This query compares players by checking when they are in the same sector at the same tic, even across different sessions.

```
SELECT
    a.player_id AS jugador_a,
    b.player_id AS jugador_b,
    COUNT(DISTINCT a.tic) AS tics_juntos
FROM TelemetryEvent a
JOIN TelemetryEvent b
  ON a.tic = b.tic
 AND a.sector_id = b.sector_id
 AND a.player_id < b.player_id
GROUP BY a.player_id, b.player_id
ORDER BY tics_juntos DESC;
```



jugador_a	jugador_b	tics_juntos
4	6	74
2	3	72
2	6	63
5	6	60
3	4	60
1	6	52
1	3	41
1	5	39
1	2	24
4	5	21
2	4	19
2	5	15

(12 rows)

Figure 7: Top co-presence results in the same sector.

Interpretation. Since the game is single-player, this does not represent real interaction. Instead, we imagine all players playing at the same time.

Higher values mean players tend to visit the same areas at similar moments in the game.

10 Indexing and performance evaluation

10.1 Important observation before indexing

The schema already creates implicit B-tree indexes for primary keys and unique constraints. In particular, `uq_telem(game_id, player_id, tic)` already creates an index over those three columns. Therefore, creating another index with exactly the same columns would be redundant. The evaluated indexes were selected to complement the existing ones.

10.2 Created indexes

Index 1: `idx_telemetry_game_tic`.
Purpose: Accelerate proximity and co-presence queries that repeatedly filter telemetry by game and tic.

```
CREATE INDEX idx_telemetry_game_tic ON TelemetryEvent(game_id, tic);
```

Index 2: `idx_telemetry_sector`.
Purpose: Improve hotspot and sector-based analyses by avoiding full-table scans when filtering specific sectors.

```
CREATE INDEX idx_telemetry_sector ON TelemetryEvent(sector_id);
```

Index 3: `idx_game_player`.
Purpose: Speed up retrieval of all participations associated with a specific player.

```
CREATE INDEX idx_game_player ON Game(player_id);
```

10.3 Before/after evidence

Index	Representative query	Buffers	Time / plan change
`idx_telemetry_sector`	Filter events with `sector_id = 655`.	493 → 3	2.07 ms → 0.069 ms; Seq Scan → Index Only Scan.
`idx_telemetry_game_tic`	Proximity inside `game_id = 1`.	168 → 150	4.9 ms → 4.7 ms; Bitmap scan becomes slightly narrower.
`idx_game_player`	Search games for `player_id = 3`.	1 → 1	About 0.02 ms → 0.02 ms; Seq Scan remains cheaper at 28 rows.

Table 1: Index evaluation using `EXPLAIN (ANALYZE, BUFFERS)`.

10.4 Discussion

Sector index. The strongest improvement came from `idx_telemetry_sector`. The filtered sector query changed from a sequential scan over the full telemetry table to an index-only scan. Buffer usage dropped from 493 to 3, and execution time dropped from approximately 2.07 ms to 0.069 ms. This index is especially useful for selective sector queries and co-presence analyses.

Game/tic index. The index `idx_telemetry_game_tic` had only a marginal improvement in the tested dataset because the unique constraint `uq_telem(game_id, player_id, tic)` already starts with `game_id`. The new index is narrower and slightly reduces buffers, but the difference is small at 34,500 rows.

Player index on Game. The index `idx_game_player` is justified by scalability. With only 28 rows in `Game`, PostgreSQL correctly chooses a sequential scan because it is cheaper than using the index. However, forcing `enable_seqscan = off` confirms that the index is usable, and it will become more valuable as the number of game participations grows.

11 C.4 Reproducibility mechanism - Makefile

To ensure full reproducibility of the database, we provide a `Makefile` that automates the process of resetting and rebuilding the entire system from scratch. This guarantees that the database can be recreated consistently using only the SQL scripts and data files included in the repository.

11.1 Makefile

```
DB_NAME=chocolate_doom
DB_USER=postgres

.PHONY: reset generate load views refresh analytics indexes test

reset:
        psql -U $(DB_USER) -c "DROP DATABASE IF EXISTS $(DB_NAME);"
        psql -U $(DB_USER) -c "CREATE DATABASE $(DB_NAME);"

generate:
        python procesar.py Telemetry.tsv \
                --game-id 1 \
                --player-id 1 \
                --map-id 1 \
                --output telemetry.tsv

load: reset
        psql -U $(DB_USER) -d $(DB_NAME) -f "DDL proyecto(1).sql"
        psql -U $(DB_USER) -d $(DB_NAME) -f "master_data.sql"
        psql -U $(DB_USER) -d $(DB_NAME) -f "parte_3_puntoB.sql"
        psql -U $(DB_USER) -d $(DB_NAME) -f "parte2_puntoB.sql"
        psql -U $(DB_USER) -d $(DB_NAME) -c "\copy staging_telemetry FROM
           'telemetry.tsv' WITH (FORMAT csv, DELIMITER E'\t', HEADER true)
           ;"
        psql -U $(DB_USER) -d $(DB_NAME) -f "etl_core.sql"
        psql -U $(DB_USER) -d $(DB_NAME) -f "C3_Views.sql"
        psql -U $(DB_USER) -d $(DB_NAME) -f "consultas_analiticas.sql"
        psql -U $(DB_USER) -d $(DB_NAME) -f "indices_evaluacion.sql"

views:
        psql -U $(DB_USER) -d $(DB_NAME) -f "C3_Views.sql"

refresh:
        psql -U $(DB_USER) -d $(DB_NAME) -c "REFRESH MATERIALIZED VIEW
           player_telemetry_summary;"

analytics:
        psql -U $(DB_USER) -d $(DB_NAME) -f "consultas_analiticas.sql"

indexes:
        psql -U $(DB_USER) -d $(DB_NAME) -f "indices_evaluacion.sql"

test:
        psql -U $(DB_USER) -d $(DB_NAME) -c "SELECT * FROM
           player_game_summary LIMIT 5;"
```

```
psql -U $(DB_USER) -d $(DB_NAME) -c "SELECT * FROM
    map_activity_summary LIMIT 5;"
```

11.2 Explanation of Makefile targets

- **reset:** This target completely reinitializes the database environment. It first drops the existing database (if it exists) and then creates a fresh empty instance. This guarantees that every execution starts from a clean and consistent state, avoiding residual data or schema conflicts from previous runs.

- **generate:** This target executes the Python preprocessing script (`procesar.py`), which transforms the raw telemetry file (`Telemetry.tsv`) into a cleaned and structured TSV file (`telemetry.tsv`). The script also assigns metadata such as `game_id`, `player_id`, and `map_id`, and computes derived fields such as sector assignment.

- **load:** This is the main orchestration target responsible for fully rebuilding the system from scratch. It first executes `reset` and `generate`, and then runs all required SQL scripts in a strict order:

  - The **DDL script**, which defines the complete database schema (tables, constraints, and relationships).
  - The **ETL script (Part 2)**, which processes and validates staging data before insertion into the final tables.
  - The **staging load step**, which imports the processed TSV file into `staging_telemetry` using PostgreSQL's `\copy` mechanism.
  - The **UX data script (Part 3)**, which inserts the GUESS-18 instrument structure and survey data.
  - The **C.3 views script**, which creates analytical views and materialized views for reporting.
  - The **analytical queries script**, which defines queries used for evaluation and insights extraction.
  - The **index evaluation script**, which creates and evaluates indexes for performance analysis.

  This ordering ensures that all dependencies are satisfied before higher-level analytical structures are built.

- **views:** Recreates only the analytical views and materialized views defined in the C.3 section without resetting or modifying the underlying data. This is useful when the database already exists but the analytical layer needs to be refreshed or updated.

- **refresh:** Recomputes the materialized view `player_telemetry_summary`, updating aggregated telemetry statistics based on the current data in the database.

- **analytics:** Executes the full set of analytical SQL queries defined in `consultas_analiticas.sql`. These queries are used to extract insights and validate the correctness of the data model.

- **indexes:** Executes the index creation and evaluation script defined in `indices_evaluacion.sql`, used to measure and demonstrate query performance improvements.

- **test:** Runs a set of lightweight validation queries over key views. These queries provide a quick sanity check to verify that the database has been correctly built and that the main analytical outputs are available.

11.3 Purpose

This automation enables the complete reconstruction of the database system through a single command, eliminating the need for manual setup steps. It guarantees a consistent and deterministic pipeline, where the same input data and scripts always produce an identical database state.

By integrating preprocessing (`generate`), schema creation, data ingestion through a staging layer, ETL processing, and analytical view construction, the Makefile ensures that the entire workflow can be reproduced end-to-end without ambiguity.

Additionally, this approach reduces the risk of human error during deployment, enforces correct execution order of dependencies, and significantly improves reproducibility across different environments, making the system easier to evaluate, test, and maintain.

12 Conclusions

In this project, we designed and implemented a relational database to store gameplay telemetry together with user information and UX survey data. The schema follows a structured approach that separates core entities such as users, games, telemetry events, and survey responses, making the system organized and scalable.

One important result is that we can connect player behavior with their subjective experience. By linking telemetry data with survey responses, we can analyze not only how players move in the game, but also how they feel about it. This allows a more complete analysis, for example comparing engagement with movement patterns and activity within the game. Additionally, even though the game is single-player, we extended the analysis to compare spatial behavior across different players.

We also considered how the data is loaded into the system. By using an ETL approach, we can take raw telemetry logs, clean them, and insert them into the database in a consistent way. This ensures the data is reliable and can be reused if we need to reload everything. The use of staging tables and error logging helps maintain data quality during ingestion.

Additionally, the use of primary keys, foreign keys, and indexes helps maintain data integrity and improves performance when running analytical queries. This is important because telemetry data grows very fast and queries need to be efficient. Views and materialized views were also implemented to simplify queries and improve performance for frequent analyses.

Overall, this project shows how a relational database can be used not only to store large amounts of data, but also to support meaningful analysis of gameplay behavior and user experience through real telemetry data and analytical queries.

A Delivered SQL files

- `ddl_corregido.sql`: corrected PostgreSQL schema.
- `parte_3_puntoB_corregido.sql`: UX instrument metadata.
- `populate_core.sql`: master data and synthetic UX responses.
- `consultas_analiticas.sql`: six analytical queries.
- `indices_evaluacion.sql`: index creation and before/after evaluation.
- `evidencia_explain_analyze.txt`: raw execution-plan evidence.